Sistemas Operativos de Tiempo Real

Técnicas Digitales III

Introducción

- Según las restricciones temporales, existen distintos tipos de aplicaciones:
  - **Ofimática**
  - Multimedia y video juegos
  - Sistemas de control

Non
Real-time

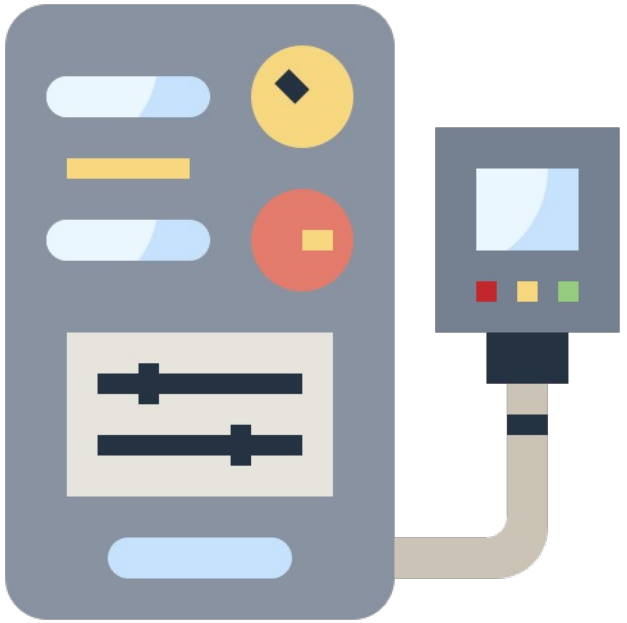
Introducción

- Según las restricciones temporales, existen distintos tipos de aplicaciones:
  - Ofimática
  - **Multimedia y video juegos**
  - Sistemas de control

Non Real-time → Soft Real-time

Introducción

- Según las restricciones temporales, existen distintos tipos de aplicaciones:
  - Ofimática
  - Multimedia y video juegos
  - **Sistemas de control**



Introducción

- Para cada una de estas aplicaciones, se utilizan distintos tipos de Sistemas Operativos
  - **Sistemas operativos de propósito general**
  - Extensiones multimedia
  - Sistema operativo de tiempo real

Introducción

- Para cada una de estas aplicaciones, se utilizan distintos tipos de Sistemas Operativos
  - Sistemas operativos de propósito general
  - **Extensiones multimedia**
  - Sistema operativo de tiempo real

Introducción

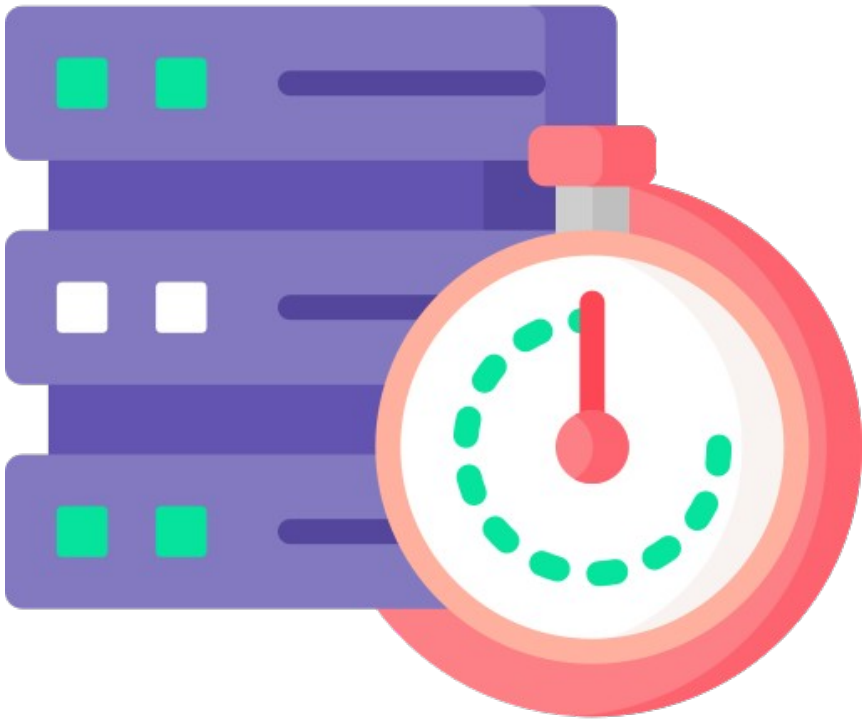
- Para cada una de estas aplicaciones, se utilizan distintos tipos de Sistemas Operativos
  - Sistemas operativos de propósito general
  - Extensiones multimedia
  - **Sistema operativo de tiempo real**

NuttX RTOS

Definición

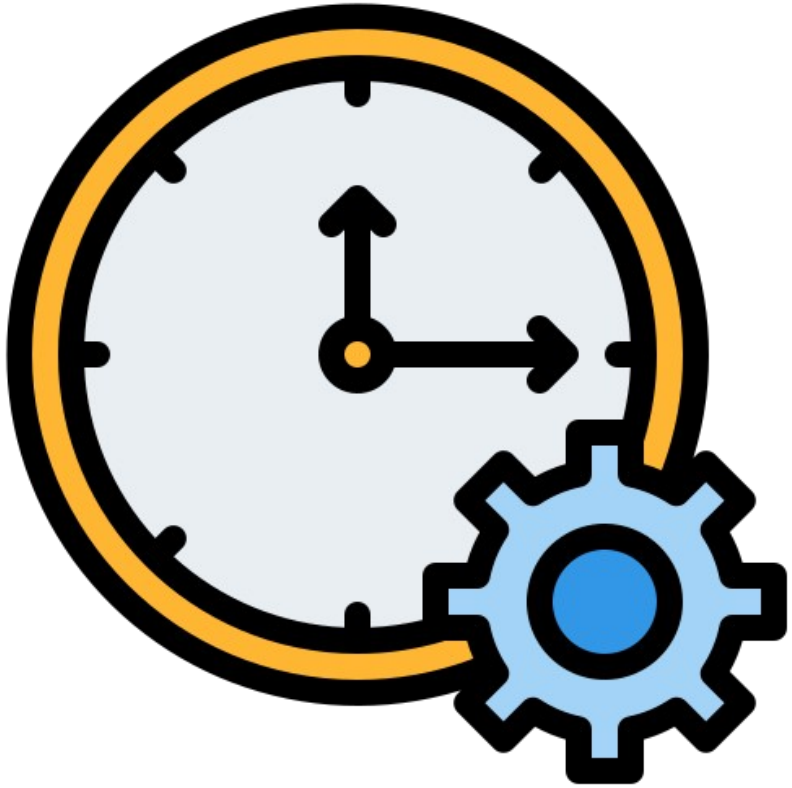
- Sistema en tiempo real:

Un sistema informático en tiempo real es aquel en el que la corrección del resultado depende tanto de su validez lógica como del instante en que se produce



Definición

- Para garantizar el tiempo de respuesta el sistema necesita **determinismo**
- Esto implica tiempos de ejecución acotados, de forma de poder garantizar **restricciones temporales**



Ejercicio

https://www.menti.com/

Código: 8562 9400

Mentimeter

Definición

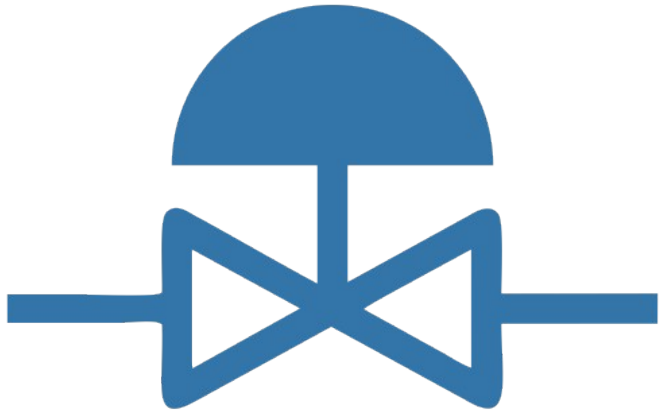
- Conviene destacar que un sistema de control en tiempo real no es necesariamente un sistema “rápido”.
- La velocidad tiene que ser tal que se ajuste a las restricciones temporales del sistema al que aplica.

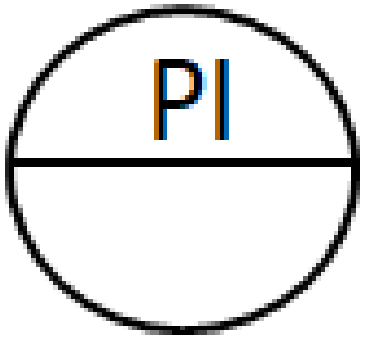
Niveles jerárquicos

- En los sistemas de control es frecuente encontrar software divido en niveles jerárquicos:
  - Adquisición de datos y actuadores
  - Algoritmos de control (PID)
  - Algoritmos de supervisión (trayectorias)
  - Interfaz de usuario, registro de datos

Niveles jerárquicos

- Adquisición de datos y actuadores:
  - Este es el nivel más bajo en la jerarquía, encargado de interactuar con el hardware del sistema a controlar.



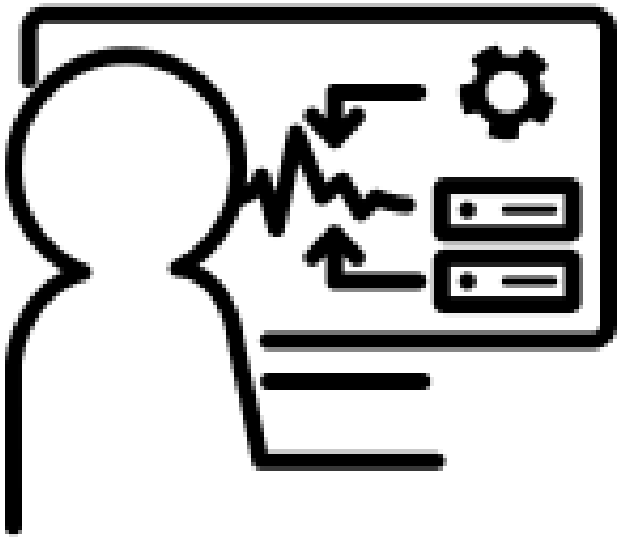


Niveles jerárquicos

- Algoritmos de control (PID):
  - Este nivel se corresponde con los algoritmos de control que, a partir de las medidas obtenidas por el sistema de adquisición de datos.

Niveles jerárquicos

- Algoritmos de supervisión:
  - En general, se encarga de supervisar el proceso y generar consignas a los controles inferiores.

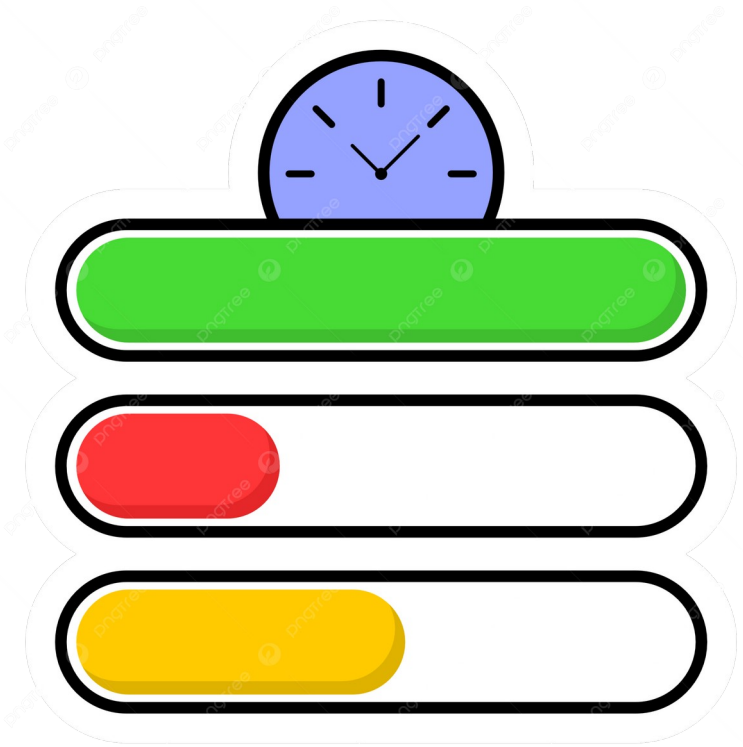


Niveles jerárquicos

- Interfaz de usuario:
  - Este nivel se corresponde con los algoritmos de control que, a partir de las medidas obtenidas por el sistema de adquisición de datos.

Tareas

- El paralelismo es una característica inherente a los sistemas en tiempo real.
- En un sistema en tiempo real el programa se divide en tareas que se ejecutan en paralelo, cada una de ellas con una determinada prioridad, para poder cumplir con sus restricciones temporales.

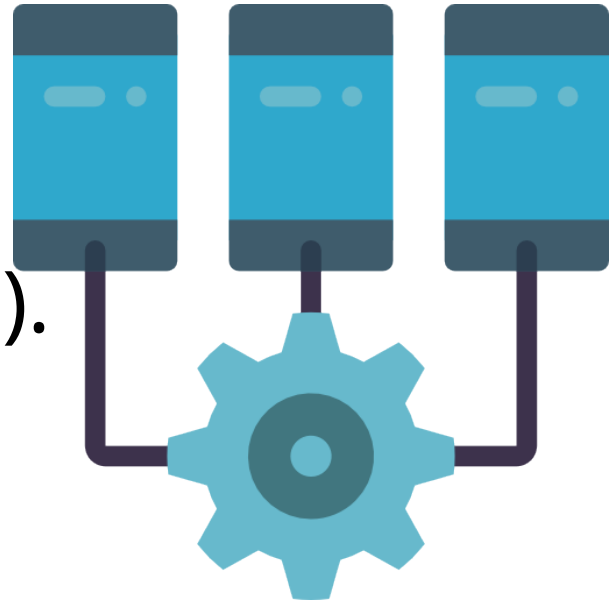


Tareas

- Salvo que se disponga de un sistema con varios procesadores, la ejecución en paralelo se consigue mediante la sucesión rápida de ejecuciones parciales de las tareas

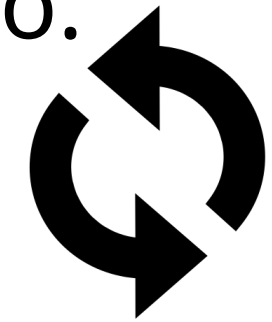
Métodos

- Existen varias técnicas de programación para ejecutar tareas en paralelo:
  - Procesamiento secuencial (Bucle de scan).
  - Primer plano / segundo plano (Foreground/Background).
  - Multitarea cooperativa.
  - Multitarea expropiativa (preemptive).



Procesamiento secuencial (scan loop)

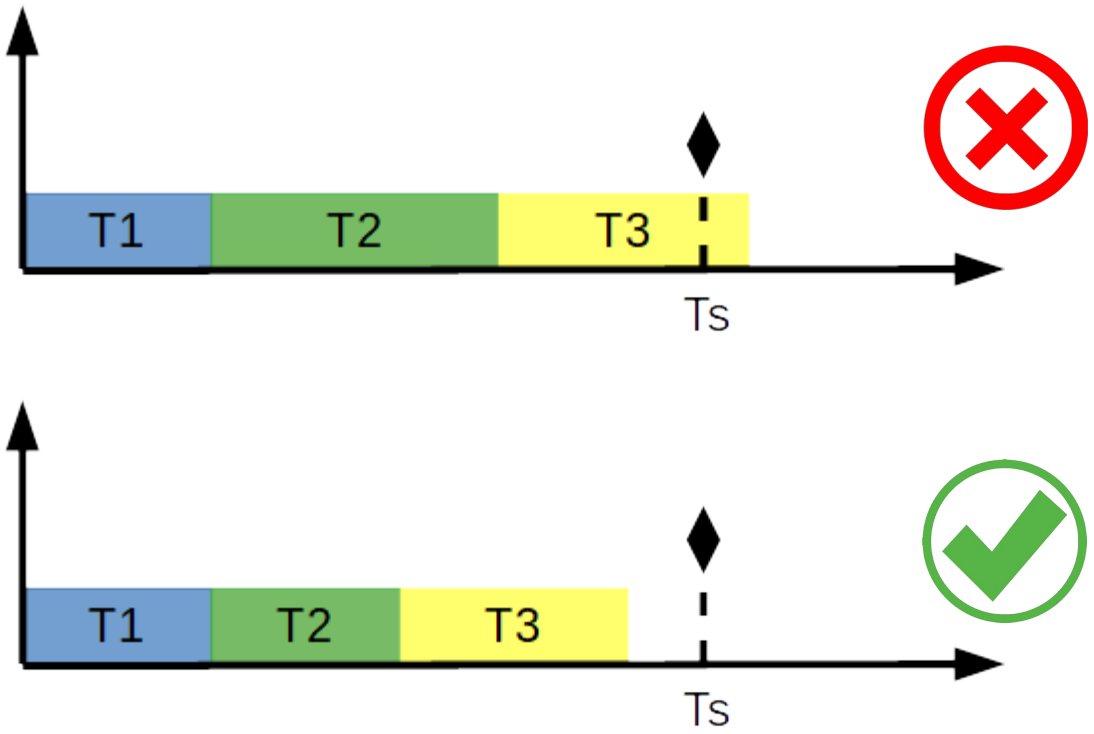
- Consiste en ejecutar todas las tareas consecutivamente una y otra vez dentro de un bucle infinito.



```
main()
{
    /* Inicialización del sistema
*/
    while(1){ /* Bucle infinito
*/
        Tarea1();
        Tarea2();
        /* ... */
        TareaN();
    }
}
```

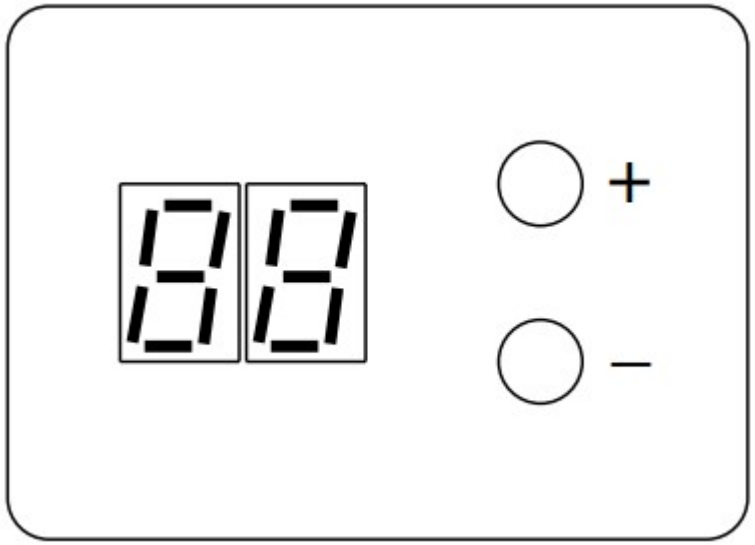
Procesamiento secuencial (scan loop)

- Condiciones:
  - Las tareas no pueden bloquearse
  - La sumatoria de los tiempos máximos de ejecución de cada tarea debe ser inferior al período de muestreo necesario



Ejemplo: Termostato

- Funcionalidad:
  - A partir de una consigna de temperatura, la cual puede ser ajustada mediante los pulsadores (+) y (-), el termostato activa o desactiva su salida para controlar un climatizador.
- Proyecto: td3_barem_01



Ejercicio

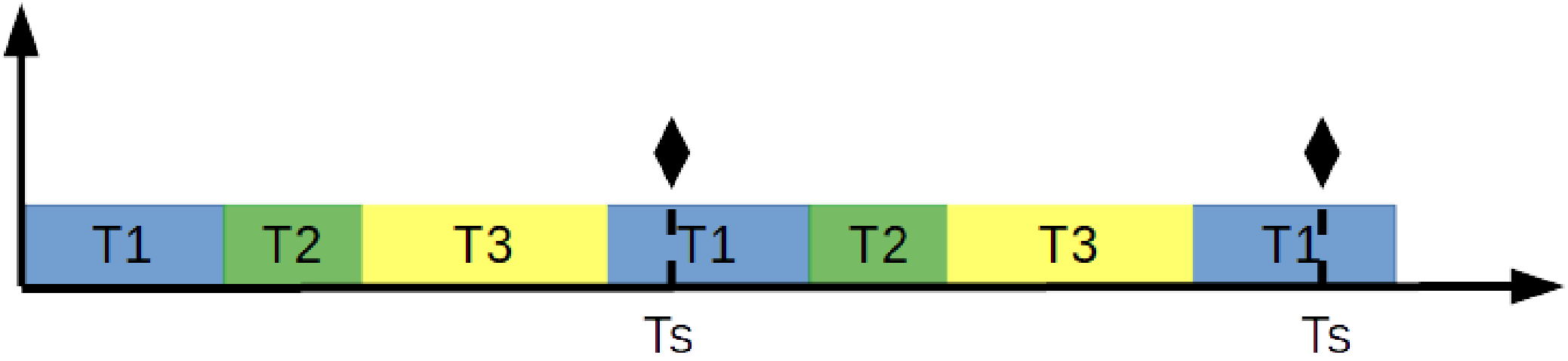
https://www.menti.com/

Código: 8562 9400

Mentimeter

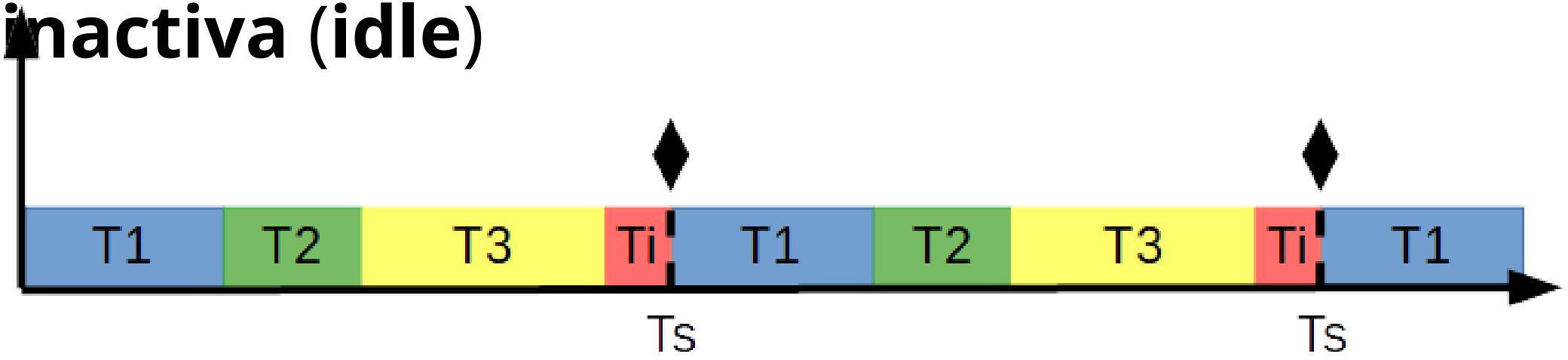
Temporización (scan loop)

- En muchos casos, se requiere un ciclo de scan (o tiempo de scan Ts)
- No siempre esta condición se cumplirá para determinadas tareas



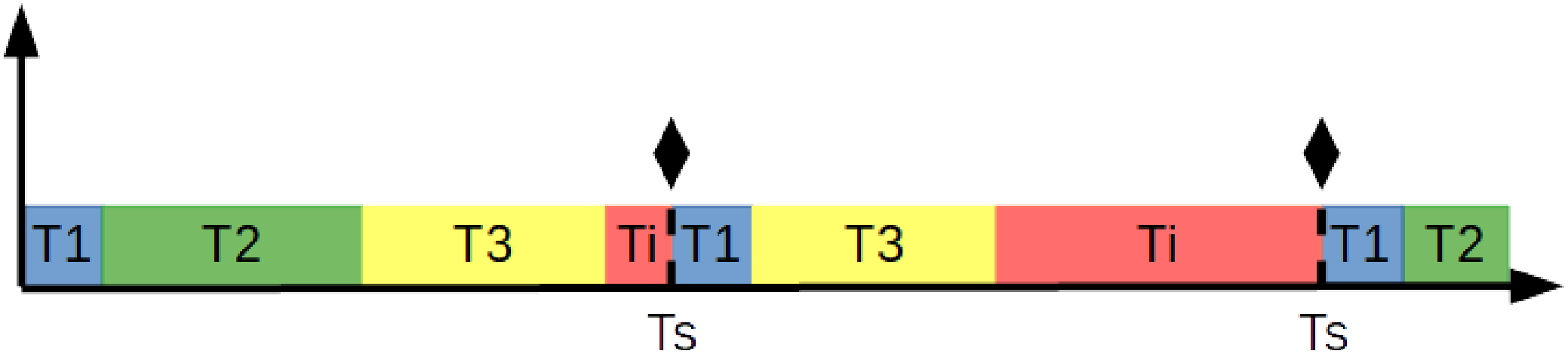
Temporización (scan loop)

- Este requerimiento se puede cumplir sumando un temporizador que ecualice la duración del ciclo.
- Esta tarea se suele denominar **ociosa o inactiva** (**idle**)



Distintos períodos (scan loop)

- Algunas tareas pueden requerir ejecutarse cada n ciclos de scan:



Distintos períodos (scan loop)

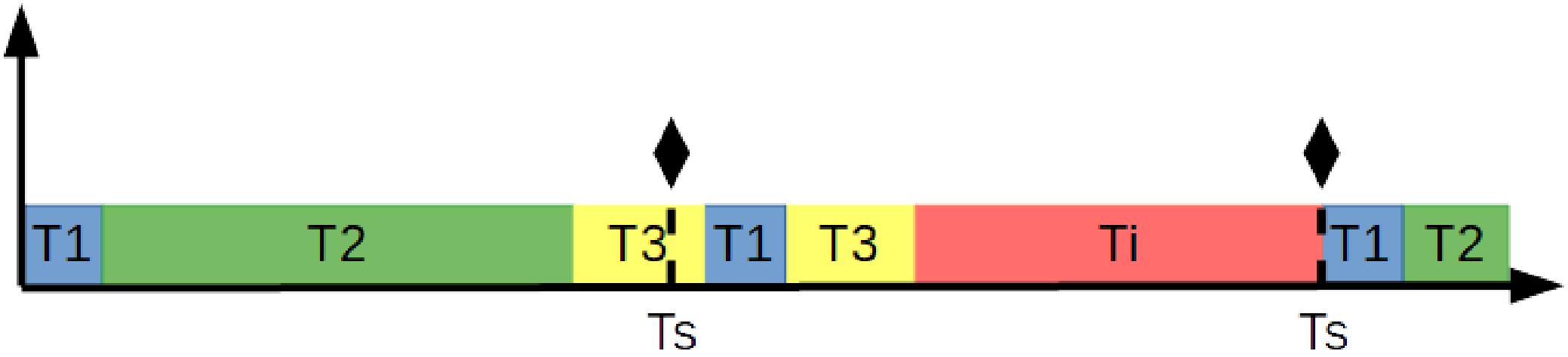
- Alternativas:

```
n=0;
for(;;){ /*for-ever*/
  TareaA();
  if(n %2==0){
    TareaB();
  }
  n++;
  TareaC();
  TareaInactiva();
}
```

```
n=2;
for(;;){ /*forever*/
  TareaA();
  if(n==2){
    TareaB();
    n=0;
  }
n++;
TareaC();
TareaInactiva();
```

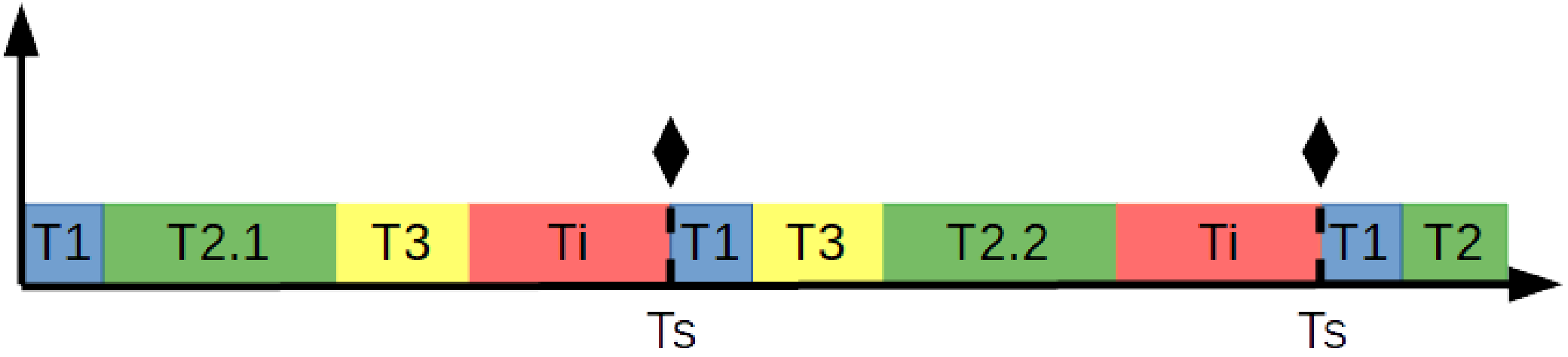
Tiempo de ejecución extenso (scan loop)

- El tiempo de ejecución excesivo representa un problema.
- Sin embargo, en ciertos casos se pueden realizar optimizaciones para el correcto funcionamiento.



Tiempo de ejecución extenso (scan loop)

- La solución más sencilla (y más costosa) es cambiar el procesador.
- En otros casos es posible dividir la tarea de larga duración en dos partes, aprovechando mejor el tiempo de procesamiento.

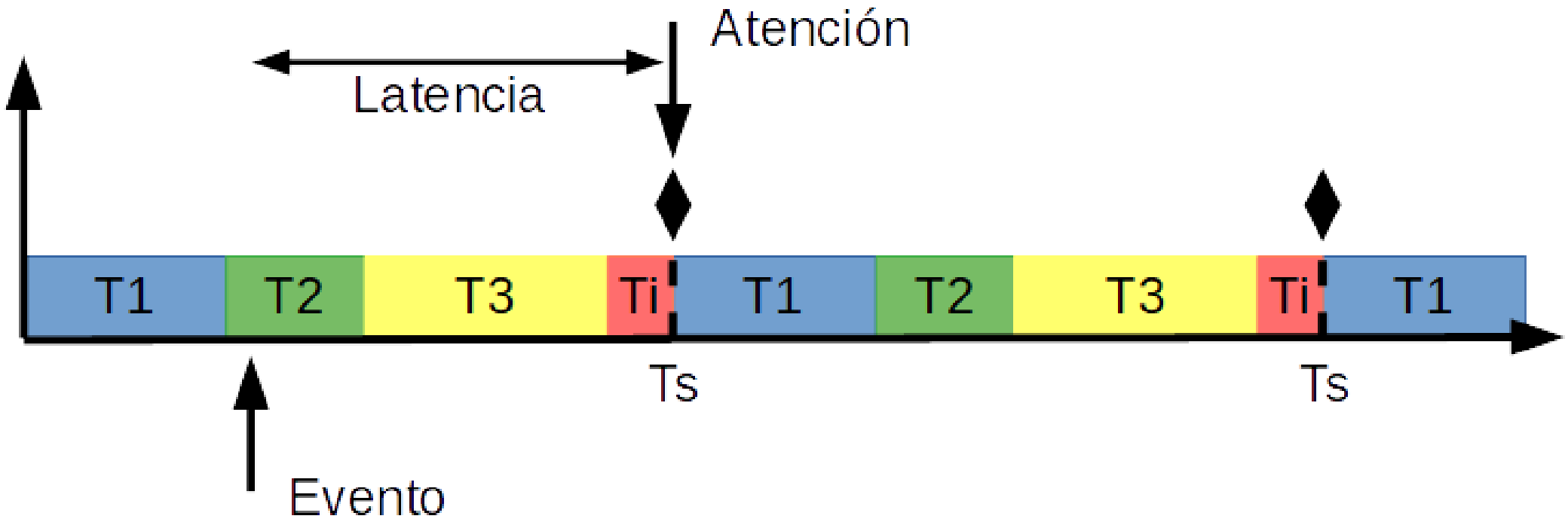


Latencia (scan loop)

La **latencia** en un sistema de tiempo real se define como el tiempo máximo que transcurre entre un evento (interno o externo) y el comienzo de la ejecución de la tarea que lo procesa.

Latencia (scan loop)

- En el procesamiento secuencial, la latencia más alta puede ser igual a Ts



Ventajas e inconvenientes (scan loop)

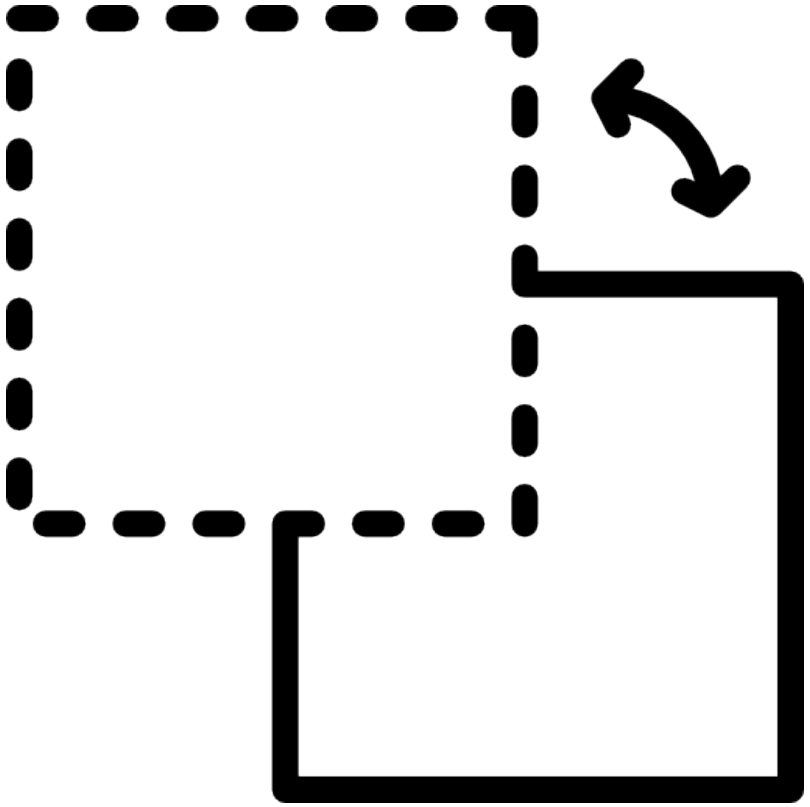
- Facilidad de implementación y depuración
- Facilidad para compartir datos
- No utiliza tiempo para cambio de contexto

Ventajas e inconvenientes (scan loop)

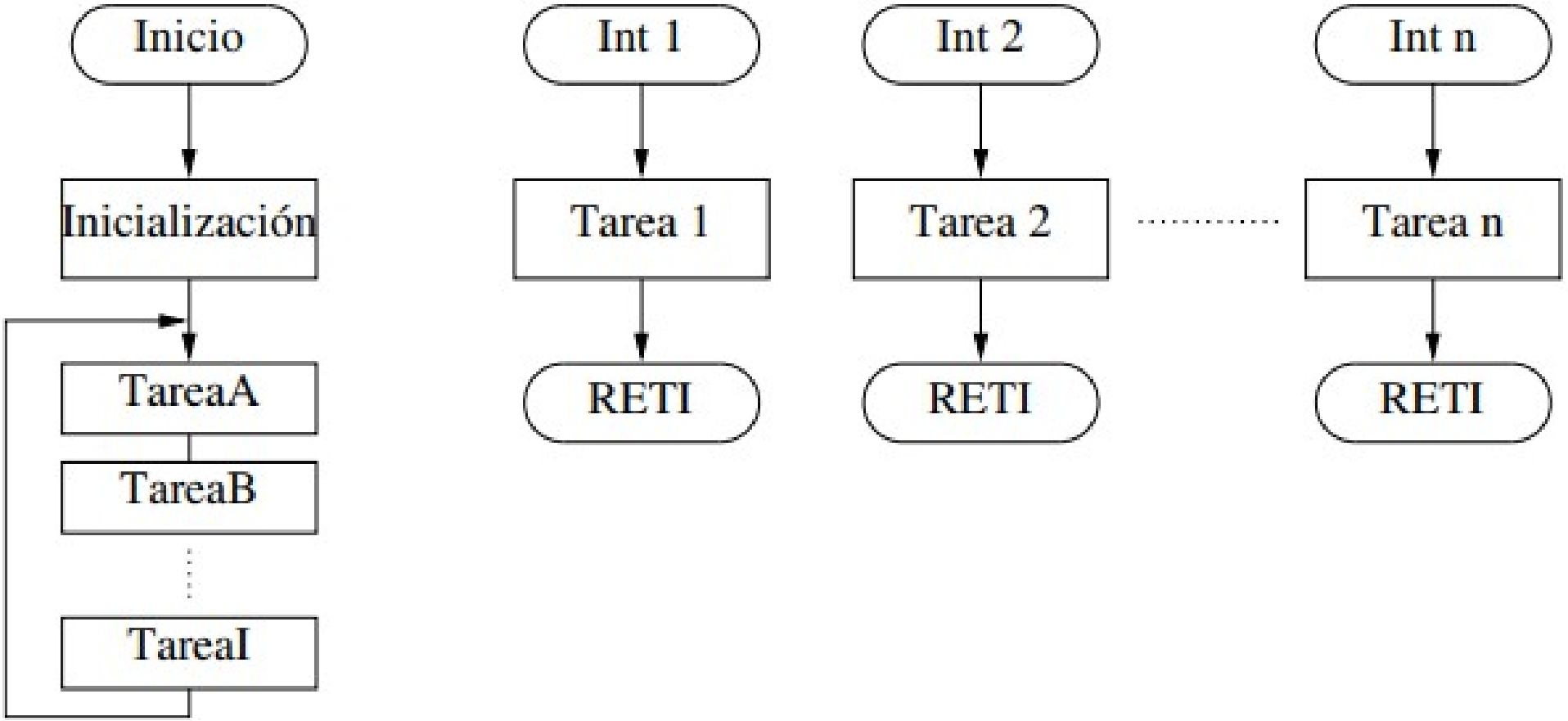
- Latencia variable
- Tareas con distintos períodos de muestreo
- No escala para sistemas complejos

Foreground/Background

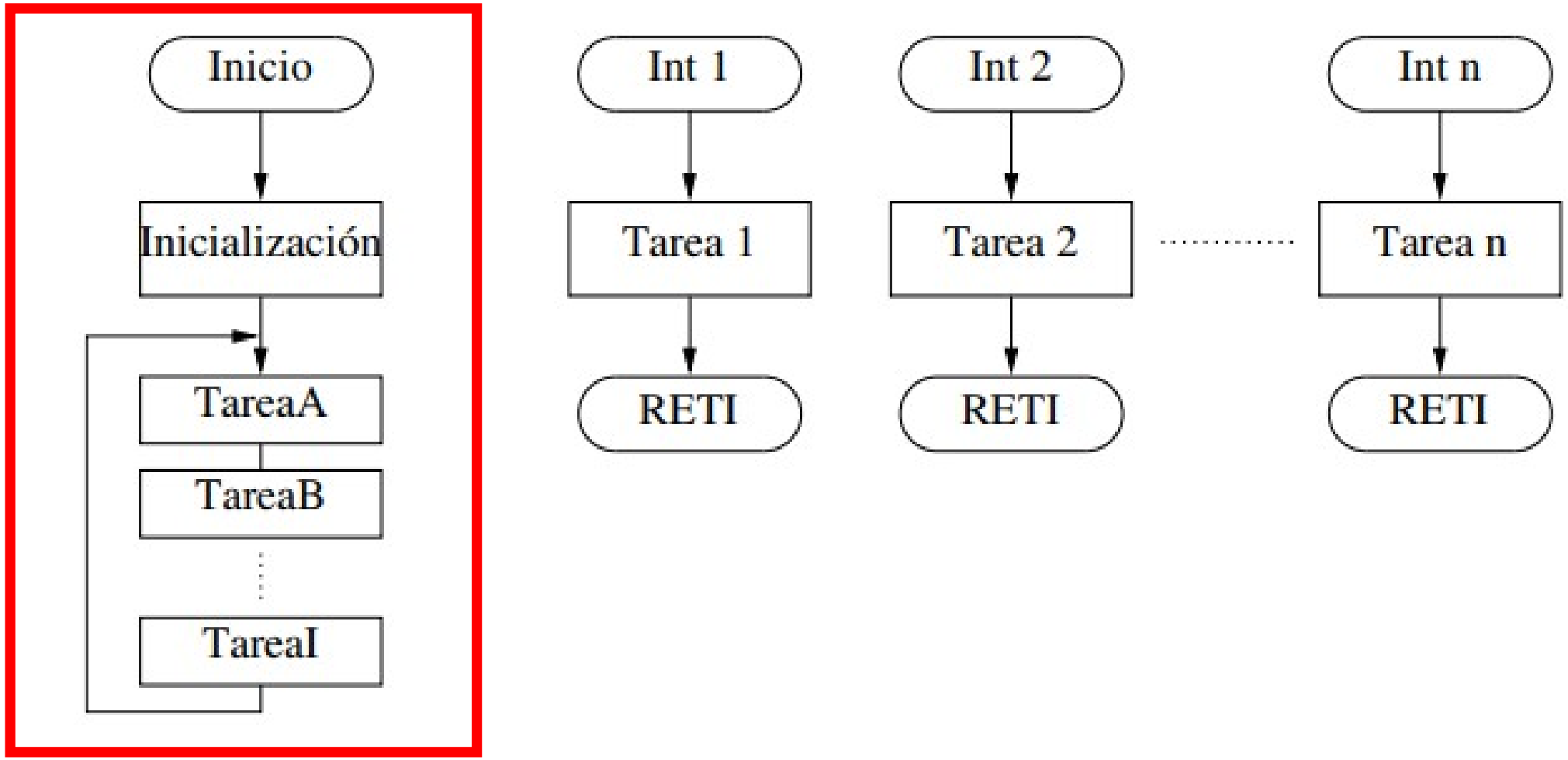
- Uno de los mayores problemas del scan loop es la latencia.
- Para disminuir este tiempo es posible asociar tareas a interrupciones.



Foreground/Background

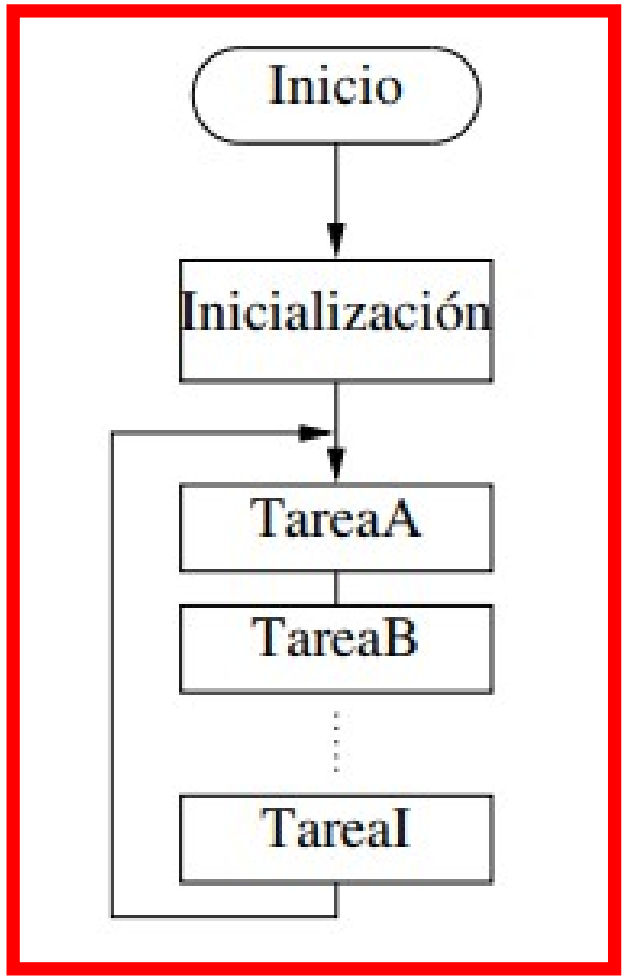


Foreground/Background

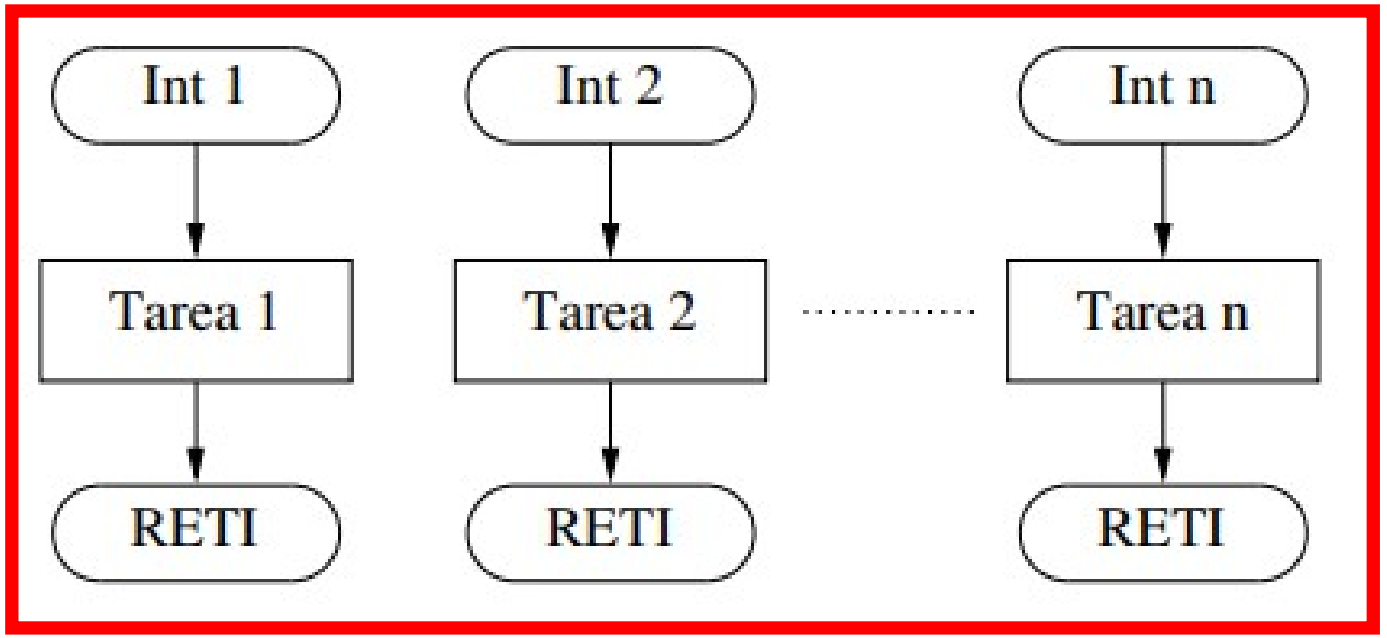


Primer Plano

Foreground/Background



Primer Plano



Segundo Plano

Foreground/Background

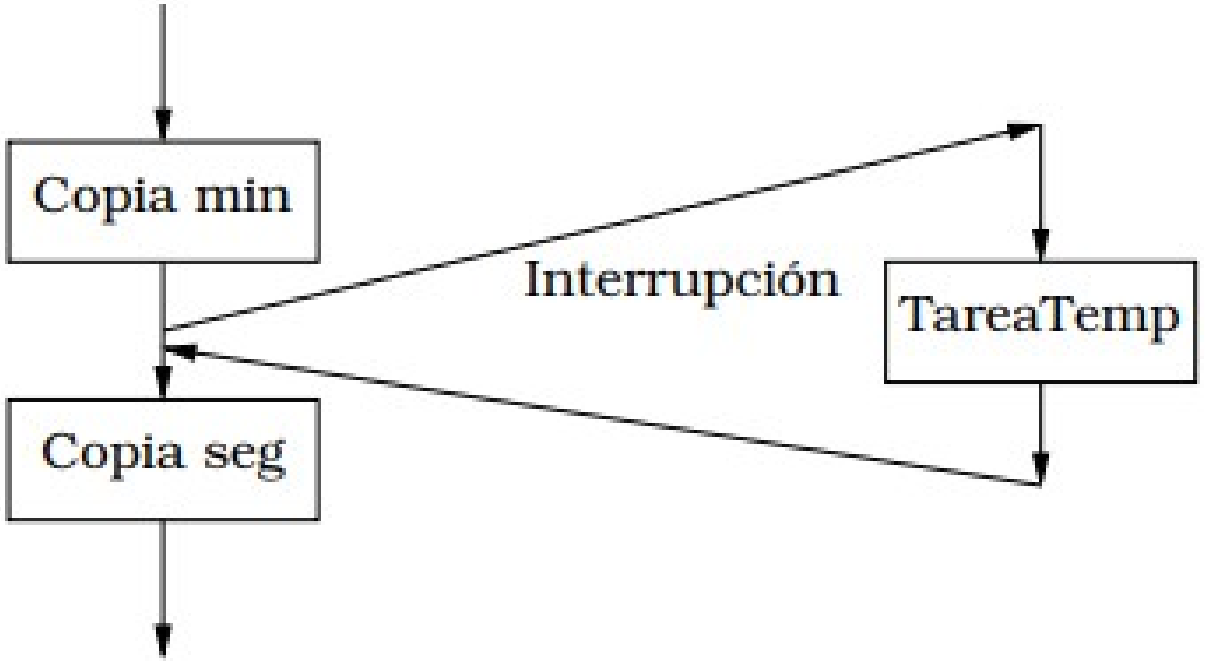
- Principales ventajas:
  - Baja latencia para la atención de tareas asociadas a interrupciones.
  - No requieren software adicional
- Inconvenientes:
  - Requieren soporte de interrupciones en el hardware
  - Cada tarea background requiere una interrupción
  - Más complejo de depurar, aparecen problemas de concurrencia

Foreground/Background

- Las tareas en segundo plano deberían tener la menor duración posible.
- Se deben trasladar al primer plano las funciones no indispensables
- Ejemplo: td3_barem_03

Foreground/Background

- Problemas de concurrencia:



Foreground/Background

- Zona crítica:
  - Se entiende por zona crítica la zona en la que se usan recursos compartidos entre dos tareas que se ejecutan de manera asincrónica.
  - Para evitar inconvenientes la ejecución de la zona crítica tiene que transitarse sin interrupción

Foreground/Background

- Zona crítica:
  - Una opción es copiar datos: td3_barem_04
  - Pero esa copia de datos también puede truncarse
  - Debo deshabilitar interrupciones durante la copia de datos: td3_barem_05

RTOS

- Para sistemas complejos:

Sistema Operativo de Tiempo Real